

Evaluating the performance of LLMs on SQL and NoSQL database using langsmith

N.Shanmugapriya¹, P.Yugeshkaran², U.Hari Charan³, M. Jayasri⁴, C. Merlyne Sandra Christina⁵, P. Anitha⁶, K. Tamilselvi⁷

¹Professor, Department of Artificial Intelligence and Data Science, Dhanalakshmi Srinivasan University, Trichy, India. shanmugapriyan.set@dsuniversity.ac.in

²B.Tech, Department of Artificial Intelligence and Data Science, Dhanalakshmi Srinivasan University, Trichy, India. yugeshkaran01@gmail.com

³B.Tech, Department of Artificial Intelligence and Data Science, Dhanalakshmi Srinivasan University, Trichy, India. haricharanuggirala1133@gmail.com

⁴Assistant Professor, Department of Computer Science and Engineering, Dhanalakshmi Srinivasan University, Trichy, India. jayasrim.set@dsuniversity.ac.in

⁵Assistant Professor, Department of Computer Science and Engineering, Dhanalakshmi Srinivasan University, Trichy, India. merlynesandrachristinac.set@dsuniversity.ac.in

⁶Assistant Professor, Department of Computer Science and Engineering, Dhanalakshmi Srinivasan University, Trichy, India. anithap.set@dsuniversity.ac.in

⁷Assistant Professor, Department of Computer Science and Engineering, Dhanalakshmi Srinivasan University, Trichy, India. tamilselvik.set@dsuniversity.ac.in

Abstract-- The integration of Large Language Models (LLMs) with organizational databases enables powerful Retrieval Augmented Generator (RAG) systems for advanced data analysis and informed decision making. Existing solutions primarily demonstrate RAG implementations with PDFs or SQL databases, often lacking a comprehensive evaluation of LLM performance. In contrast, this research aims to identify the most effective RAG system for both SQL and NoSQL databases by evaluating the performance of twelve leading LLMs from OpenAI, Meta AI, Google AI and Anthropic. The evaluation leverages LangSmith to assess performance across key metrics such as accuracy, correctness, error rate, P50 latency and P99 latency, Ultimately proposing the best suited LLM model for RAG based database applications

Keywords: LLMs, RAG, database, SQL, LangSmith, OpenAI, Meta AI, Google AI, Anthropic, accuracy, correctness, error rate and latency.

I INTRODUCTION

Decision making is key for every organization, as it depends on data sources from the database. When an industry relies on a database for making decisions, analyzing large datasets becomes challenging due to their vast size. It requires a large team to perform preprocessing, advanced analytical skills, and complex query processing to retrieve data. To overcome these challenges, we integrate Large Language Models (LLMs) with the database system, enabling proficient communication without the need for SQL and Document query languages.

A. Purpose

The purpose of this research is not just to develop a database RAG system but to primarily focus on identifying the best LLMs for both SQL and NoSQL databases. This is achieved by evaluating their performance using various metrics such as accuracy, correctness, error rate, P50 latency and P99 latency.

B. Objective

- Advance analytics
- Reduce cost and time
- Reduce man power
- NLP driven database
- Increase productivity

II. SOFTWARE TOOLS AND TECHNOLOGIES

A. Python 3.12.4

This Retrieval Augmented Generation (RAG) system is built using the python programming language, which offers a rich set of AI and Natural Language Processing (NLP) libraries for developing LLM based applications efficiently. Notable libraries include LangChain, LlamaIndex, Google Generative AI, OpenAI and Hugging Face, which enable smooth integration of generative AI capabilities. Additionally, Python provides easy connectivity with both SQL database (MySQL,

PostgreSQL, SQLite) and NoSQL databases (MongoDB, DynamoDB, Elasticsearch, Firebase). It also includes efficient data retrieval tools such as FAISS, an opensource vector search library designed for fast and scalable similarity searches in high dimensional spaces.

B. LangChain

Langchain is a python library that serves as a core package for building Retrieval Augmented Generation (RAG) application. It enables AI powered SQL query generation and document query formulation, allowing users to interact with databases using natural language. Langchain automatically converts user queries into SQL or document query languages based on the database type and schema. By leveraging LLMs, ChatPromptTemplate and database integration, langchain dynamically extracts schema information to generate accurate queries. Additionally, it integrates with langsmith to evaluate the accuracy and quality of the responses generated.

C. LangSmith

LangSmith is used to evaluate the performance of production grade LLM applications. In this RAG system, it is integrated to assess performance based on key metrics such as semantic accuracy, response correctness, error rate, P50 latency and P99 latency, comparing the ground truth values with the obtained responses. By leveraging LangSmith, we can analyze and compare the performance of multiple LLM models within the RAG system, helping to identify the best performing model for optimal results.

D. Flask

Flask is a python backend framework used in this RAG application to handle HTTP requests, process user queries, interact with the database, and manage responses between the database RAG system and the frontend.[17] It provides a lightweight and flexible REST API, allowing users to send natural language queries via POST requests to the /query-db endpoint. Flask process these queries by invoking the LangChain based SQL or document query generation pipeline, executing the generated queries on MySQL or MongoDB and returning the formatted response. Additionally, flask maintains conversation history to ensure context aware responses. It also supports Cross Origin Resource Sharing (CORS), enabling efficient communication between the frontend and backend. In this RAG system, flask plays a crucial role in integrating the evaluation module, receiving input queries, processing them through LangChain and facilitating interaction with LangSmith. Overall, flask act as a bridge between the user, LangChain, LangSmith and

the database, ensuring smooth communication and efficient query processing.

E. Database

This RAG system is designed to support both SQL and NoSQL databases. The application architecture is separately built for SQL databases and NoSQL databases, utilizing different LLMs to each database type.

1. MySQL Database:

This application serves as the primary data source, where all structured information is stored and retrieved. It contains various tables with relevant business or institutional data that users can query using natural language. When a user submits a query, the system powered by the LangChain generates a corresponding SQL query, which is then executed on the MySQL database to fetch the required data. The database plays a crucial role in maintaining data integrity, consistency and accessibility, ensuring that the responses generated by LLM models are based on real time up to date information. Additionally, the SQLDatabase utility from LangChain enables dynamic schema extraction, allowing LLM models to understand the database structure and generate precise queries accordingly.

2. MongoDB Database:

MongoDB serves as the primary database for storing and retrieving JSON formatted data in this RAG system. It is integrated using the MongoDBDatabase class, which connects to the MongoDB Atlas cluster via a connection string. This application utilizes MongoDB to store JSON data and provides a key functionality. The ability to dynamically generate and execute MongoDB queries based on user questions using an AI driven generator. This process leverages LangChain and LLM models to analyze user queries, generate executable MongoDB queries and retrieve relevant data efficiently.

F. Visual Studio Code Editor

Visual Studio Code is a powerful code editor used for developing and managing this database RAG application. It provides an efficient coding experience with features such as syntax highlighting and debugging tools, making it easier to write, test and maintain python code. The integrated termina; in VS Code allows developers to run the flask application, install dependencies and execute MongoDB & SQL queries efficiently. Additionally, its built in support for virtual environment (venv) and docker helps in managing dependencies and streamlining deployment processes. VS Code also supports a provides a thunder client extension which is a alternative tools like postman, allowing developers to send HTTP requests directly from VS

Code without switching to an external application. With thunder client, developers can test various API endpoints by making GET, POST, PUT and DELETE requests, providing live responses from the flask backend. It supports authentication methods, headers, query parameters and body formats like JSON and form data, making it ideal interacting with the MongoDB and MySQL database through API calls.

III. RELATED WORKS

[1] Xiang Zhang, Khatoon Khedri and Reza Rawassizadeh conducted a study to evaluate the resource usage and accuracy of none LLM models incuding LLaMA 2 7B, LLaMA 2 13B, Mistral, Optimus-7B, SUS-Chat-34B, PlatypusYi-34B, NeuralHermes-2.5-Mistral-7B and Starling-LM-7B-Alpha for generating natural language queries in comparison to traditional SQL. The study advises against replacing relational databases with LLMs due to high resource demands and significant energy. [2] Xuanhe Zhou, Zhaoyan Sun and Guoliang Li has proposed the DB-GPT framework for fine tuning and designing LLMs specifically for databases to address challenges such as prompt design, lack of physical database, awareness and privacy concerns. [3] Zijin Hong and his team conducted a survey on using LLM models to convert natural language questions into SQL. While pretrained model have improved accuracy but challenges remains with complex queries and schema. [4] Johan Sandell and his team conducted research on comparing relational databases such as ArangoDB, MySQL and Neo4j, focusing on metrics like energy consumption, CPU usage and memory usage. Results shows Neo4j outperforms MySQL and ArangoDB in handling connected graph data efficiently and better for big data application. [5] Mahdhindi and his team conducted a survey on the use of RAG in the legal domain, highlighting its ability to enhance LLM outputs by retrieving

external knowledge. The survey reviews RAG methods, their stages, evaluation metrics, relevant datasets and legal specific applications. Identified challenges include high computational costs, complex in-house setups and retrieval inefficiencies. The accuracy of retrieval heavily affects response quality, especially due to complex legal language and lack of domain adapted models. [6] Binita Saha and his team introduced QulM-RAG, an improved RAG system for domain specific question answering utilizing a custom dataset and the LLaMA-3-8B-Instruct model. It reduces hallucination and improves accuracy compared to traditional RAG systems. [7] Rong Hu and his team introduces a Intelligence Custom Clearance Assistant (ICCA) RAG system which is used for custom clearance

tasks. It integrated multimodal document parsing, hybrid storage and context aware generation to handle document complexity, query ambiguity and dynamic regulations. ICCA-RAG improves accuracy, contextual relevance and robustness to noisy queries. [8] Luis Bernardo Hernandez Salinas and his team proposed intelligent driving assistant system (IDAS) using AI and RAG to enable multilingual voice based interaction with vehicles. It effectively answers user queries using car manual, with GPT-4o showing the best performance and smaller models like GPT-4o mini offering faster responses. Challenges in real-time communication due to lack of database integration to track live data.

[9] Nouf Alabbasi and his team introduces TeleOracle which is a telecom specific RAG system built on phi-2 model. It uses a two stage retriever with semantic chunking and hybrid search, plus expanded context window for better query handling. TeleOracle achieves 81.2% accuracy, a 30% improvement over base phi2 and performance comparably to large LLM with higher faithfulness to context. It faces limitations in handling non textual data (eg., tables, graphs) and complex reasoning tasks. [10] Gulsum Budaglu and Hakan Emekci perform evaluation on RAG system, fine-tuning LLM and hybrid of both models on the datasets like Stanford Question Answering (SQuAD) and Microsoft Machine Reading Comprehension (MS MARCO) and SQL create table statements. Fine-tuning offers strong semantic accuracy but uses heavy resource, while RAG is efficient in resource usage but less precise. The hybrid model balances both, achieving the highest semantic and contextual performance, especially on structured data. However, it is also a resource intensive.

[11] Yosua Setyawan Soekamto and his team proposed a SKYRAG (Separated Keyword Retrieval Augmentation Generation) a keyword based RAG system that integrates LLMs to generate personalized learning paths by retrieving relevant Massive Open Online Course (MOOC) content. It outperforms naïve (traditional) RAG in accuracy, relevance and learner satisfaction across multiple domain. Future plans include real-time integration, multi-language support and advanced learning analytics to enhance scalability and adaptability in personalized education. However, the study does not provide details on the integration of the database with the SKYRAG system, nor does it specify the type of database employed, such as whether it utilizes a SQL based or NoSQL based architecture.

[12] Menna Fateen and her team introduces a modular RAG based system for automatic short answer scoring with ASAS-F (Automatic Short Answer Scoring - Feedback) using ColBERT, which improves scoring and feedback generation without heavy fine-tuning or manual prompt engineering.

Using RAG for few shot selection and automatic prompt generation. While it reduces dependence on expensive fine-tuning. Feedback generation quality still lags behind in BLEU and ROUGE score compared to fine-tuned approaches. However, the paper does not describe how the underlying data storage or database system is integrated with RAG system. [13] Jun Xu and his team introduces ChatTf, a knowledge graph enhanced Q&A system designed to reduce hallucination in domain like Chinese folklore. It uses custom ontology Traditional Folklore Ontology (TFOnto), a triple extraction framework Zero-Shot Traditional Folklore Triplet Extraction (ZFTE) and build a knowledge graph – Traditional Folklore Knowledge Graph (TFKG). A dual stage TFKG-RAG framework integrates this knowledge with LLMs to improve accuracy. ChatTf achieved 96.7% accuracy on a folklore Q&A test set, surpassing existing methods. Challenges includes lack of integration of folklore databases which limits intelligent connectivity and effective knowledge retrieval.

[14] Majjed AL-Qatf and his team proposes RAG data space models (RAD-DSMs), integrating RAG with data spaces to improve access to diverse, high quality external knowledge while maintaining secure data sharing. The unified lifecycle aims to

enhance both LLM performance and data space services. Despite its potential, the approach faces challenges in data quality, privacy and system integration. [15] Alexander Tobias Neumann and his team proposed MoodleBot, and LLM based chatbot integrated into Moodle to support self regulated learning using the RAG approach and evaluated via the Technology Acceptance Model (TAM), it showed 88% accuracy and positive student feedback. Limitations, the study only used GPT-4, lacking comparison with other LLMs. [16] Shijun Ge and team have proposed a study showing that combining large language models (LLMs) with traditional concept generation methods (like brainstorming and TRIZ) using chain-of-thought (CoT) techniques improves conceptual design. The findings confirm that integrating LLMs with structured design methods is effective, suggesting broader future use in design fields. [18] António Mestre and his team have proposed an architecture to implement RAG effectively using relational databases as data sources, but they did not propose an architecture for NoSQL databases. [19] Sergey Vichev and team explores improving Text-to-SQL tasks using a RAG approach by dynamically augmenting LLM prompts with database context.

IV. PROPOSED SYSTEM

System Architecture

1. MySQL RAG System

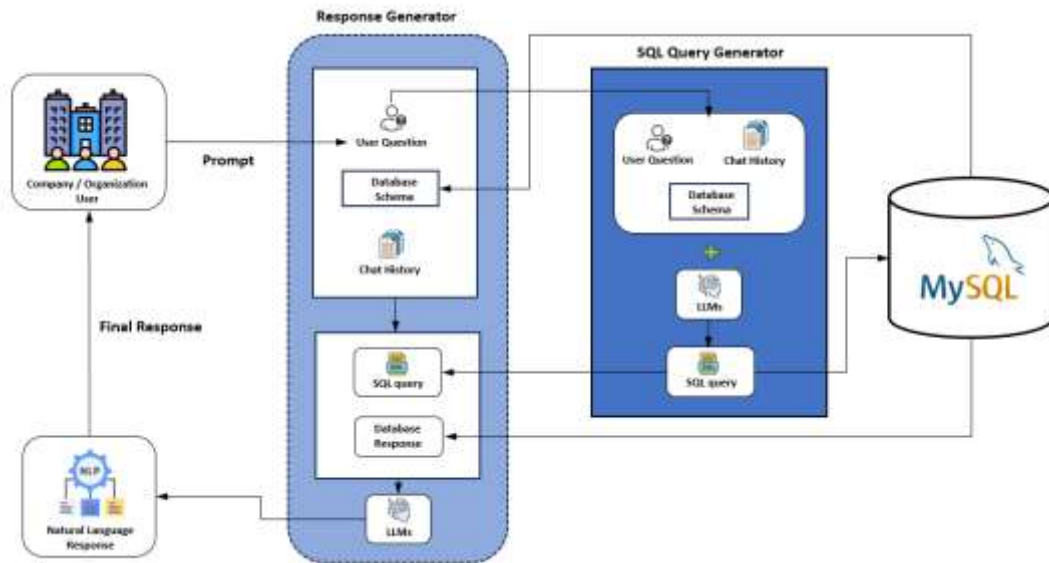


Fig. 1 MySQL RAG Architecture

The proposed solution consists of three main components namely **response_generator**, **db_query_generator** and **MySQL database**. The **response_generator** serves as the primary component of the RAG system. It is connected to the database schema using the **SQLDatabase** utility from LangChain, along with the chat_history from previous conversations. When a user submits a question, the **reponse_generator** requires a corresponding SQL query to retrieve data from the database. To generate this SQL query, the **response_generator** communicates with the **db_query_generator** component, passing parameters such as the database schema, user question and chat history. The **db_query_generator** then utilizes an LLM model to generate the appropriate SQL query and returns it to the **response_generator**. The **response_generator** executes the generated SQL query on the database and retrieves the results. Finally, the **response_generator** uses the LLM model to formulate a natural language response based on the database result, chat history and user question. This response is then returned to the user as the final answer.

Example:

User Question: Give me student name list

Db_query_generator : SELECT name FROM students;

2. MongoDB RAG System

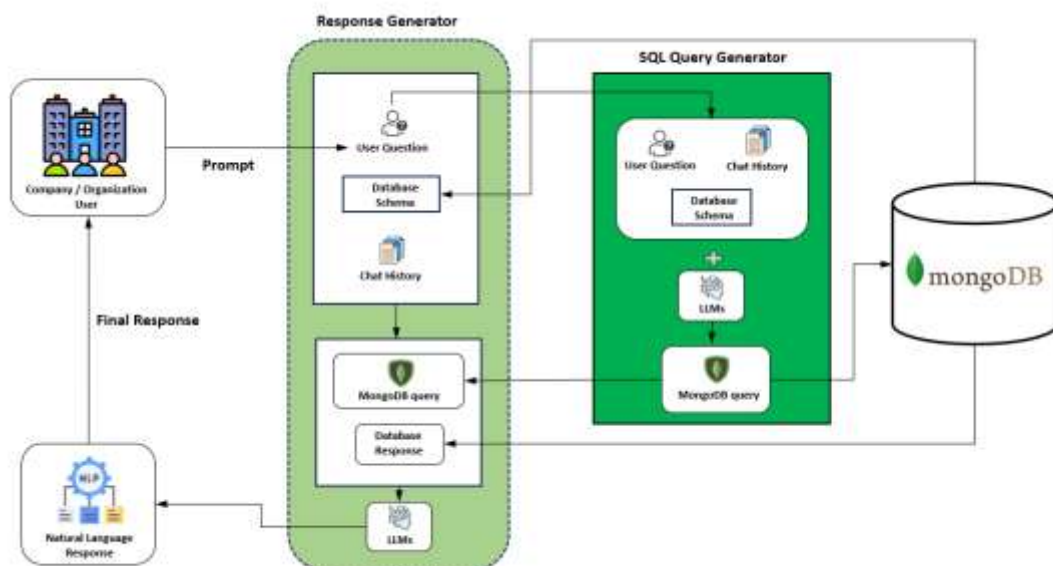


Fig. 2 MongoDB RAG Architecture

The proposed solution for the MongoDB based RAG system closely follows the architecture of the MySQL based RAG system, with the key difference being the use of MongoDB as the database and a

Response_generator : The list of students are a, b,c,d,e,.etc.

Example 2:

User Question: How many faculty members are working at the college?

Db_query_generator: SELECT COUNT(*) AS total_faculty FROM faculty_tbl;

Response_generator : There are totally 16 members are working in the college.

Example 3:

User Question: Student with more than 80% of mark?

Db_query_generator : select * from students where Mark >80;

Response_generator : The students who scored more than 80% marks are A, B, C, D and M.